

PROYECTO FINAL DE CICLO

2º DAM — NortempoFormación

EduIA

Asistente Educativo Inteligente con RAG para el Ciclo DAM

Autor:

Brayan González Cid

Centro: NortempoFormación

10 de Junio 2026

Tecnologías: Java Spring Boot · Python FastAPI · React · PostgreSQL · LangChain · OpenAI

Resumen

EduIA es un asistente educativo inteligente desarrollado como proyecto final del ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM). La aplicación integra un backend en Java con Spring Boot, un servicio de inteligencia artificial en Python con FastAPI y LangChain, y un frontend en React. Su objetivo principal es proporcionar a los estudiantes del ciclo DAM un chatbot especializado capaz de responder preguntas técnicas apoyándose en documentación propia subida por el usuario mediante un sistema RAG (Retrieval-Augmented Generation).

El sistema implementa autenticación segura con JWT, soporte multiusuario concurrente, historial de conversaciones persistente en PostgreSQL, subida y procesamiento de documentos PDF y TXT, exportación de conversaciones y estadísticas de uso. La arquitectura sigue un diseño en tres capas desacopladas que se comunican mediante API REST.

El resultado es una plataforma funcional, escalable y extensible que demuestra la integración de tecnologías modernas de desarrollo web con capacidades de inteligencia artificial generativa.

Palabras clave: inteligencia artificial, RAG, chatbot educativo, Spring Boot, FastAPI, React, LangChain, JWT, PostgreSQL.

1. Motivación e Introducción

1.1 Motivación

La rápida evolución de la inteligencia artificial generativa, especialmente con la irrupción de modelos de lenguaje de gran tamaño (LLMs), ha abierto nuevas posibilidades en el ámbito educativo. Sin embargo, la mayoría de los asistentes IA genéricos no están especializados en contenidos técnicos concretos, lo que limita su utilidad para estudiantes de programación.

Durante el transcurso del ciclo DAM, los estudiantes se enfrentan a una gran cantidad de tecnologías, frameworks y conceptos que deben asimilar en poco tiempo. Disponer de un asistente especializado, capaz de responder preguntas concretas sobre los temarios y los proyectos del ciclo, puede suponer una ventaja significativa en el proceso de aprendizaje.

La motivación principal de este proyecto es, por tanto, construir una herramienta práctica que combine las tecnologías aprendidas durante el ciclo —desarrollo backend, frontend y bases de datos— con las capacidades actuales de la IA, aplicándolas a un problema real: el apoyo al aprendizaje técnico.

1.2 Contexto y Problema

El aprendizaje técnico en ciclos formativos de informática presenta varios desafíos: la diversidad de tecnologías a dominar, la dificultad de acceder a documentación específica de forma rápida .

Los chatbots de uso general, aunque potentes, presentan limitaciones: no tienen acceso a los materiales específicos del centro, responden con información genérica que puede no adaptarse al nivel del ciclo, y no mantienen el contexto de las conversaciones del alumno a lo largo del tiempo.

EduIA resuelve estos problemas permitiendo que los propios alumnos suban sus apuntes y documentos, de modo que el sistema pueda responder preguntas contextualizadas en su propio material de estudio.

1.3 Objetivos

Los objetivos del proyecto son:

- Desarrollar una aplicación web multiusuario con autenticación segura basada en JWT.
- Implementar un sistema RAG completo que permita responder preguntas basadas en documentos subidos por el usuario.
- Integrar un servicio de IA externo (OpenAI) mediante una arquitectura de microservicios.
- Gestionar conversaciones con historial persistente en base de datos relacional.
- Proporcionar una interfaz de usuario intuitiva y funcional desarrollada en React.
- Demostrar la viabilidad de integrar tecnologías modernas de IA con un stack de desarrollo web convencional.

2. Metodología, Recursos y Planificación

2.1 Metodología

El desarrollo de EduIA ha seguido una metodología iterativa e incremental inspirada en los principios ágiles. El proyecto se dividió en sprints de trabajo semanales, comenzando por la arquitectura base y añadiendo funcionalidades de forma progresiva.

Las fases principales del desarrollo fueron:

1. Diseño de arquitectura y definición de requisitos.
2. Configuración del entorno: Spring Boot, PostgreSQL, React.
3. Implementación del sistema de autenticación JWT.
4. Desarrollo del servicio IA con FastAPI y LangChain.
5. Implementación del sistema RAG para documentos.

6. Integración de todos los componentes y pruebas.
7. Refinamiento de la interfaz y corrección de errores.

Este enfoque permitió detectar y resolver problemas de integración entre los tres servicios de forma temprana, reduciendo el riesgo de fallos en las etapas finales.

2.2 Estimación de Recursos

Los recursos utilizados en el proyecto son:

Tipo de Recurso	Detalle	Coste Estimado
Hardware	PC de desarrollo (CPU i5, 16 GB RAM)	0 € (propio)
Software	Eclipse IDE, VS Code, Postman, pgAdmin	0 € (gratuito)
Servicios cloud	OpenAI API (GPT-3.5-turbo)	~5-10 € (uso en desarrollo)
Tiempo	Aprox. 150 horas de desarrollo	—
Documentación	Lectura de docs, tutoriales	0 €

2.3 Planificación

La planificación aproximada del proyecto en semanas fue:

Semana	Actividad Principal	Estado
1-2	Diseño de arquitectura, modelo de datos, wireframes	✓ Completado
3-4	Backend Spring Boot: auth, modelos, repositorios	✓ Completado
5-6	Servicio IA: FastAPI, LangChain, integración OpenAI	✓ Completado
7-8	Sistema RAG: carga de documentos, embeddings, consultas	✓ Completado
9-10	Frontend React: login, chat, historial, subida docs	✓ Completado
11-12	Integración, pruebas, corrección de errores	✓ Completado
13	Redacción de la memoria	✓ En curso

3. Tecnologías y Herramientas

3.1 Backend — Java Spring Boot

Spring Boot 3.x es el framework principal del backend. Proporciona inyección de dependencias, gestión de la capa de persistencia mediante Spring Data JPA, y configuración automática que reduce el tiempo de arranque del proyecto. Se eligió por ser una tecnología ampliamente usada en el mercado laboral y por la solidez de su ecosistema.

3.2 Servicio IA — Python FastAPI

FastAPI es un framework moderno de Python orientado a la creación de APIs REST de alto rendimiento. Se utiliza para alojar el servicio de inteligencia artificial, separándolo del backend principal para mayor modularidad. Su integración nativa con tipos Python y su documentación automática (Swagger) facilitan el desarrollo.

3.3 LangChain y OpenAI

LangChain es una librería de Python que facilita la construcción de aplicaciones basadas en modelos de lenguaje. En EduIA se utiliza para implementar el sistema RAG: carga de documentos, generación de embeddings, almacenamiento en vector store (FAISS) y recuperación de contexto relevante para cada pregunta. Los embeddings y la generación de respuestas se realizan mediante la API de OpenAI con el modelo GPT-3.5-turbo.

3.4 Frontend — React

React es la librería JavaScript elegida para el frontend. La aplicación está estructurada en componentes funcionales con hooks, y se comunica con el backend mediante peticiones Axios. Las páginas principales son: Login, Registro y Chat (que incluye historial de conversaciones y subida de documentos).

3.5 Base de Datos — PostgreSQL

PostgreSQL es el sistema gestor de bases de datos relacional utilizado. Se accede mediante Spring Data JPA con Hibernate como ORM. Las tablas principales son: users, conversaciones, mensajes y documentos.

3.6 Autenticación — JWT

La autenticación se implementa mediante JSON Web Tokens (JWT). Al iniciar sesión, el servidor genera un token firmado que el cliente incluye en el encabezado Authorization de cada petición. El

filtro de seguridad valida el token en cada request antes de acceder a los endpoints protegidos.

3.7 Otras Herramientas

Eclipse IDE: entorno de desarrollo para el backend Java.

VS Code: editor para Python y React.

Postman: pruebas manuales de la API REST.

pgAdmin: gestión visual de la base de datos PostgreSQL.

Git / GitHub: control de versiones del proyecto.

4. Análisis

4.1 Requisitos Funcionales y No Funcionales

Requisitos Funcionales

- RF01: El sistema debe permitir el registro de nuevos usuarios con nombre, email y contraseña.
- RF02: El sistema debe permitir el inicio de sesión y gestionar la sesión mediante JWT.
- RF03: Los usuarios autenticados pueden iniciar nuevas conversaciones con el asistente IA.
- RF04: Cada conversación almacena el historial completo de mensajes en base de datos.
- RF05: El sistema permite subir documentos en formato PDF y TXT para enriquecer el contexto de la IA.
- RF06: El sistema RAG utiliza los documentos subidos para generar respuestas contextualizadas.
- RF07: El usuario puede eliminar conversaciones del historial.
- RF08: El usuario puede exportar una conversación completa en formato JSON.
- RF09: El sistema ofrece estadísticas del usuario (nº de conversaciones, mensajes) y globales.
- RF10: El sistema dispone de un Data Seeder con usuarios de ejemplo para facilitar las pruebas.

Requisitos No Funcionales

- RNF01: El sistema debe soportar múltiples usuarios concurrentes con sesiones independientes.

RNF02: Las contraseñas deben almacenarse hasheadas (BCrypt) en la base de datos.

RNF03: La comunicación entre el frontend y el backend debe realizarse mediante HTTPS en producción.

RNF04: El tiempo de respuesta del chatbot no debe superar los 15 segundos en condiciones normales.

RNF05: El código debe estar estructurado en capas bien definidas (MVC + servicios).

RNF06: El sistema debe manejar errores de comunicación entre servicios de forma controlada.

4.2 Modelo de Datos

La base de datos PostgreSQL contiene las siguientes tablas principales:

Tabla	Campos Principales	Descripción
users	id, nombre, email, password, rol, fecha_registro	Usuarios registrados en el sistema
conversaciones	id, titulo, fecha_creacion, user_id (FK)	Conversaciones de cada usuario
mensajes	id, contenido, rol (user/assistant), timestamp, conversacion_id (FK)	Mensajes de cada conversación
documentos	id, nombre, ruta, fecha_subida, user_id (FK)	Documentos subidos para el RAG

Las relaciones son: un usuario tiene muchas conversaciones (1:N); una conversación tiene muchos mensajes (1:N); un usuario tiene muchos documentos (1:N).

4.3 Casos de Uso

Los principales casos de uso del sistema son:

Caso de Uso	Actor	Descripción Resumida
CU01: Registrarse	Usuario anónimo	El usuario crea una cuenta proporcionando nombre, email y contraseña.
CU02: Iniciar sesión	Usuario registrado	El usuario se autentica y recibe un token JWT.
CU03: Enviar pregunta	Usuario autenticado	El usuario escribe una pregunta y el sistema responde con ayuda de la IA.
CU04: Subir documento	Usuario autenticado	El usuario sube un PDF o TXT para que la IA lo use como contexto.
CU05: Ver historial	Usuario autenticado	El usuario consulta sus conversaciones anteriores.
CU06: Eliminar conversación	Usuario autenticado	El usuario elimina una conversación del historial.

CU07: Exportar conversación	Usuario autenticado	El usuario descarga una conversación en formato JSON.
CU08: Ver estadísticas	Usuario autenticado	El usuario consulta sus estadísticas y las globales del sistema.

5. Diseño

5.1 Arquitectura del Sistema

EduIA sigue una arquitectura de tres capas desacopladas que se comunican mediante API REST:

Frontend React (puerto 3000): interfaz de usuario, gestiona el estado de la sesión y se comunica con el backend a través de Axios.

Backend Java Spring Boot (puerto 8080): lógica de negocio, autenticación JWT, acceso a base de datos y comunicación con el servicio IA.

Servicio IA Python FastAPI (puerto 8000): recibe las preguntas del backend, ejecuta el sistema RAG con LangChain y devuelve la respuesta generada por OpenAI.

El flujo típico de una petición es: el usuario escribe una pregunta en el frontend → el frontend envía la petición al backend con el token JWT → el backend valida el token, guarda el mensaje en PostgreSQL y reenvía la pregunta al servicio IA → el servicio IA ejecuta RAG (recupera contexto de los documentos) y llama a OpenAI → la respuesta vuelve al backend, que la persiste y la devuelve al frontend.

5.2 Diagrama UML de Clases

Las principales clases del backend Java son:

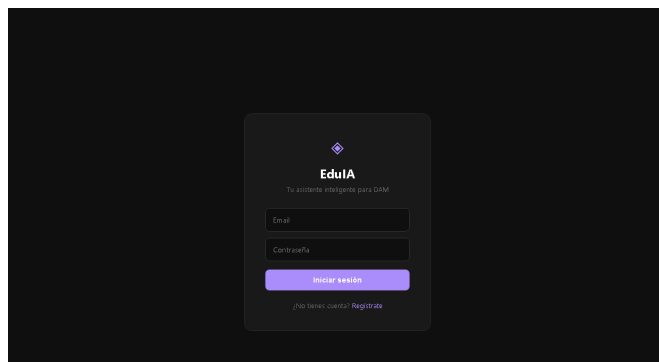
Clase / Entidad	Tipo	Atributos Principales	Relaciones
User	Entidad JPA	id, nombre, email, password, rol	1:N con Conversacion, 1:N con Documento
Conversacion	Entidad JPA	id, titulo, fechaCreacion, user	N:1 con User, 1:N con Mensaje
Mensaje	Entidad JPA	id, contenido, rol, timestamp, conversacion	N:1 con Conversacion
Documento	Entidad JPA	id, nombre, ruta, fechaSubida, user	N:1 con User

ServicioAuth	Service	—	Usa UsuarioRepositorio, JwtUtil
ServicioChat	Service	—	Usa MensajeRepositorio, ServicioIACliente
ServicioIACliente	Service / HTTP Client	—	Llama al servicio Python en puerto 8000
JwtUtil	Componente	secretKey, expiration	Usado por FiltroJwt y ServicioAuth
FiltroJwt	OncePerRequestFilter	—	Valida token en cada petición

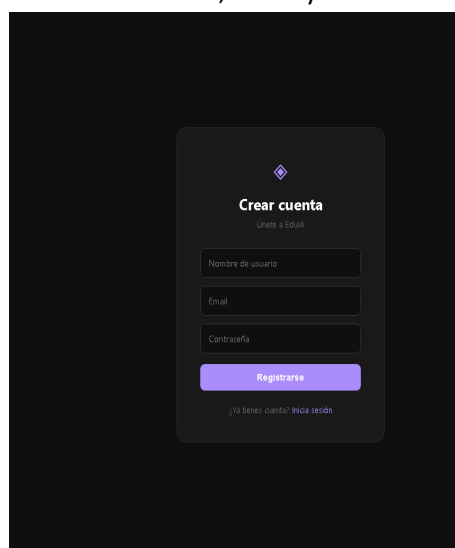
5.3 Mockups de Interfaz

La interfaz se compone de tres vistas principales:

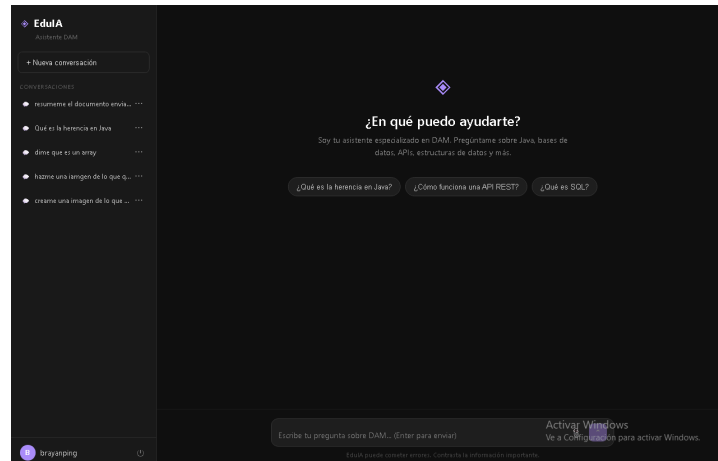
Pantalla de Login: formulario centrado con campos email y contraseña, enlace a registro.



Pantalla de Registro: formulario con nombre, email y contraseña con validación básica.



Pantalla de Chat: panel lateral con historial de conversaciones y botones de acción (nueva conversación, eliminar, exportar); área principal de chat con burbuja de mensajes diferenciada por usuario/asistente; barra inferior con input de texto, botón de subida de documentos y botón de envío.



5.4 Decisiones de Diseño

Se separó el servicio IA del backend principal para aislar las dependencias de Python del proyecto Java, facilitando el mantenimiento y la escalabilidad independiente de cada componente.

Se eligió FAISS como vector store en memoria para simplificar el despliegue en entorno local, siendo posible sustituirlo por un vector store persistente (como Chroma o Pinecone) en un despliegue en producción.

El título de cada conversación se genera automáticamente a partir de la primera pregunta del usuario, eliminando la necesidad de que el usuario nombre manualmente cada sesión.

6. Implementación

6.1 Estructura del Proyecto

El proyecto se organiza en tres directorios principales dentro del workspace:

edu-ia-backend/: proyecto Maven de Spring Boot con la estructura estándar src/main/java.

edu-ia-service/: proyecto Python con FastAPI, LangChain y el sistema RAG.

edu-ia-frontend/: aplicación React generada con Create React App.

Dentro del backend Java, los paquetes se organizan en: config, controller, dto, model, repository, security y service. Esta separación en capas garantiza el principio de responsabilidad única y facilita las pruebas.

6.2 Componentes Principales

Backend — Autenticación JWT

La clase JwtUtil genera y valida tokens JWT firmados con HMAC-SHA256. El FiltroJwt intercepta cada

petición HTTP, extrae el token del encabezado Authorization, lo valida y carga el contexto de seguridad de Spring. El endpoint `/api/auth/login` devuelve el token al cliente tras verificar las credenciales con BCrypt.

Servicio IA — Sistema RAG

El gestor_ia.py implementa el pipeline RAG: carga los documentos del usuario desde disco con DocumentLoader, los divide en chunks con RecursiveCharacterTextSplitter, genera embeddings con OpenAIEmbeddings y los indexa en un FAISS vector store. Al recibir una pregunta, recupera los chunks más relevantes mediante similitud coseno y los incluye en el prompt enviado a GPT-3.5-turbo.

Frontend — Gestión del Estado

El componente Chat.js gestiona el estado de la sesión mediante React hooks. Mantiene la lista de conversaciones, los mensajes de la conversación activa y el estado de carga. Utiliza useEffect para cargar el historial al iniciar sesión y actualiza el estado tras cada respuesta de la IA.

6.3 Dificultades y Soluciones

Dificultad Encontrada	Solución Adoptada
CORS entre los tres servicios en puertos distintos	Configuración explícita de CorsConfig en Spring Boot y cabeceras CORS en FastAPI para permitir el origen del frontend.
Manejo de errores cuando el servicio IA no está disponible	El backend captura las excepciones de comunicación HTTP y devuelve un mensaje de error descriptivo al frontend en lugar de un 500 genérico.
Contexto de documentos compartido entre usuarios	Cada usuario tiene su propio subdirectorio de documentos y su propio vector store FAISS, garantizando el aislamiento entre sesiones.
Persistencia del historial en conversaciones largas	Cada mensaje se persiste en la tabla mensajes en el momento en que se genera, evitando pérdida de datos si la sesión se interrumpe.

7. Multiusuario Concurrente

El soporte multiusuario es uno de los requisitos obligatorios del proyecto y está implementado en varios niveles del sistema.

7.1 Aislamiento de Sesiones con JWT

Cada usuario que inicia sesión recibe un token JWT único firmado con una clave secreta del servidor. Este token incluye el email y el identificador del usuario en su payload. Todas las peticiones protegidas requieren este token en el encabezado Authorization. De este modo, dos usuarios conectados simultáneamente tienen tokens diferentes y el backend identifica inequívocamente a qué usuario pertenece cada petición.

7.2 Gestión de la Concurrencia en Spring Boot

Spring Boot crea un hilo de ejecución por cada petición HTTP entrante gracias al servidor embebido Tomcat. El contexto de seguridad (SecurityContextHolder) es local al hilo, por lo que no existe riesgo de que la sesión de un usuario se mezcle con la de otro. El acceso a la base de datos se gestiona mediante un pool de conexiones (HikariCP por defecto en Spring Boot), que distribuye eficientemente las conexiones entre los hilos concurrentes.

7.3 Aislamiento de Documentos por Usuario

El servicio IA mantiene un vector store FAISS separado por usuario, indexado por su identificador único. Cuando el servicio RAG recibe una pregunta, carga únicamente el índice del usuario que realizó la petición, garantizando que los documentos de un usuario no son accesibles ni influyen en las respuestas de otro usuario.

7.4 Prueba de Concurrencia

Se verificó el funcionamiento multiusuario abriendo dos sesiones simultáneas en navegadores diferentes con los usuarios de ejemplo proporcionados por el Data Seeder. Ambas sesiones mantuvieron historiales independientes y recibieron respuestas correctas sin interferencias.

8. Despliegue e Instalación

8.1 Manual de Instalación

Requisitos previos:

- Java 17 o superior y Maven instalados.
- Python 3.10 o superior con pip.
- Node.js 18 o superior con npm.
- PostgreSQL 14 o superior en ejecución local.

Pasos de instalación:

1. Clonar el repositorio: `git clone https://github.com/brayanping/IA_TFG.git`
2. Crear la base de datos en PostgreSQL: `CREATE DATABASE eduia;`
3. Configurar `application.properties` en el backend con las credenciales de la base de datos y la clave JWT.
4. Crear el fichero `.env` en `edu-ia-service` con la clave `OPENAI_API_KEY`.

5.Instalar dependencias Python: `cd edu-ia-service && pip install -r requirements.txt`

6.Instalar dependencias React: `cd edu-ia-frontend && npm install`

8.2 Configuración

El backend se configura mediante el fichero `application.properties`. Los parámetros más importantes son:

`spring.datasource.url`: URL de conexión a PostgreSQL (por defecto `jdbc:postgresql://localhost:5432/eduia`).

`spring.datasource.username / password`: credenciales de la base de datos.

`jwt.secret`: clave secreta para firmar los tokens JWT.

`ia.service.url`: URL del servicio Python (por defecto `http://localhost:8000`).

El servicio IA se configura en `config.py` con la URL base y en el fichero `.env` con la clave de OpenAI.

8.3 Datos de Ejemplo

El Data Seeder (`DataSeeder.java`) se ejecuta automáticamente al arrancar el backend y crea los siguientes usuarios de ejemplo si la tabla está vacía:

Email	Contraseña	Rol
profesor@dam.com	123456	ADMIN
alumno@dam.com	123456	USER

8.4 Arranque del Sistema

El orden correcto de arranque es:

- Backend Spring Boot: desde Eclipse, click derecho en `EduIABackendApplication.java` → Run As → Spring Boot App. O con Maven: `mvn spring-boot:run`.
- Servicio IA Python: `cd edu-ia-service && venv\Scripts\activate && uvicorn main:app --reload --port 8000`
- Frontend React: `cd edu-ia-frontend && npm start` (o doble click en `arrancar.bat`).

8.5 Manual de Usuario

Una vez el sistema está en marcha, se accede desde el navegador en `http://localhost:3000`. El usuario puede registrarse o iniciar sesión con las credenciales de ejemplo. Desde la pantalla de chat puede escribir preguntas en el campo inferior, subir documentos con el botón de adjunto, navegar por el historial de conversaciones en el panel lateral, y exportar o eliminar conversaciones mediante los botones de acción.

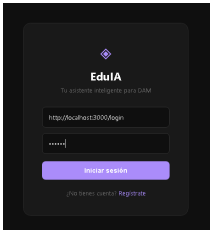
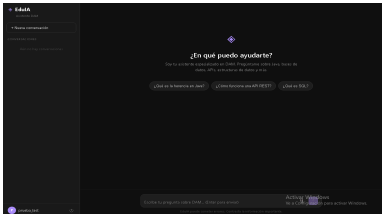
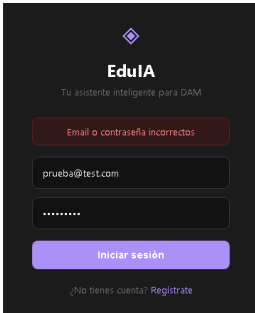
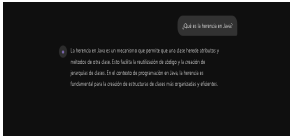
9. Pruebas

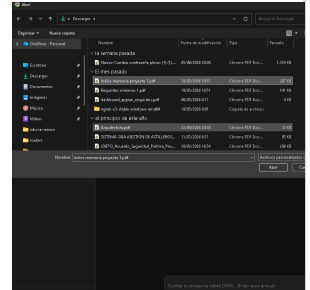
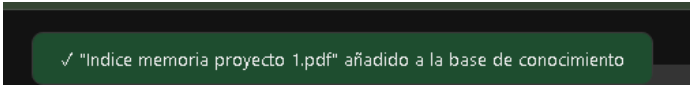
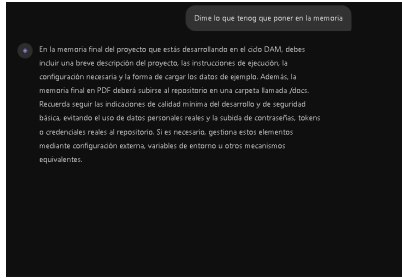
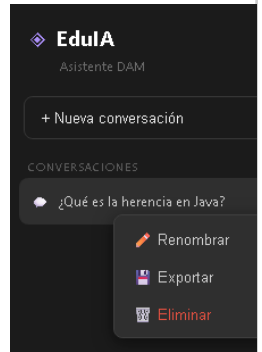
9.1 Plan de Pruebas

El plan de pruebas cubre las funcionalidades principales del sistema: autenticación, gestión de conversaciones, sistema RAG, gestión de documentos y estadísticas. Las pruebas se realizaron de forma manual mediante Postman para los endpoints del backend y pruebas de integración completa a través del frontend.

9.2 Pruebas

ID	Descripción	Precondición	Pasos
P1	Registro de usuario nuevo	Sistema en marcha, email no registrado	Registrarse con los datos correspondientes. Completado vamos al chat.

P2	Login con credenciales válidas	Usuario registrado en BD	Iniciamos sesión con las credenciales oportunas.  Nos lleva al chat. 
P3	Login con contraseña incorrecta	Usuario registrado en BD	Ponemos mal la contraseña. 
P4	Envío de mensaje al chat	Usuario autenticado, servicio IA activo	Le preguntamos a la IA. 

P5	Subida de documento PDF	Usuario autenticado	Hacer click en el clip de abajo a la derecha.  
P6	Consulta RAG con documento subido	Usuario con documento PDF subido	Pregunta relacionada con el contenido del PDF 
P7	Eliminación de conversación	Usuario con al menos una conversación	Vamos a una conversación y vemos tres puntos no ponemos encima y clicamos en eliminar. 

			EduIA Asistente DAM + Nueva conversación CONVERSACIONES Aún no hay conversaciones								
P8	Acceso sin token JWT	Sistema en marcha	Vamos postman ponemos la url que vemos en pantalla y veremos como salta un error. POST http://localhost:3030/api/chat/completions Send Params Authorization Headers Body Script Tests Settings Query Params <table><thead><tr><th>Key</th><th>Value</th><th>Description</th><th>Bulk Edit</th></tr></thead><tbody><tr><td>key</td><td>value</td><td>description</td><td></td></tr></tbody></table> Body Cookies Headers Test Results 403 Forbidden 7 ms 339 B Try with different auth credentials	Key	Value	Description	Bulk Edit	key	value	description	
Key	Value	Description	Bulk Edit								
key	value	description									

10. Resultados, Conclusiones y Vías Futuras

10.1 Resultados

El proyecto EduIA ha alcanzado todos los objetivos planteados inicialmente. Se ha desarrollado una aplicación web completamente funcional que integra tres servicios independientes comunicados mediante API REST, con autenticación JWT, sistema RAG operativo y soporte multiusuario concurrente.

El sistema es capaz de responder preguntas técnicas de forma contextualizada basándose en documentos subidos por el usuario, lo que lo diferencia de un chatbot genérico y le otorga un valor práctico real para el estudio del ciclo DAM.

10.2 Conclusiones

Este proyecto ha demostrado que es viable integrar tecnologías modernas de IA generativa (LLMs,

RAG) dentro de una arquitectura de software convencional de ciclo formativo, sin necesidad de recursos de computación especiales ni infraestructura costosa.

Desde el punto de vista técnico, el proyecto ha permitido consolidar conocimientos en múltiples tecnologías: Spring Boot, Spring Security, JWT, JPA/Hibernate, Python asíncrono, LangChain, React y PostgreSQL.

La mayor dificultad encontrada fue la coordinación entre tres servicios en distintos lenguajes y puertos, lo que requirió una planificación cuidadosa de la API y la gestión de errores entre servicios.

10.3 Vías Futuras

Entre las posibles mejoras y extensiones del proyecto se identifican:

- Despliegue en la nube (AWS, Azure o Render) con Docker y CI/CD.

- Sustitución de FAISS en memoria por un vector store persistente (Chroma, Pinecone) para no perder los índices al reiniciar el servicio.

- Soporte para más formatos de documento: DOCX, HTML, Markdown.

- Implementación de un sistema de roles más avanzado: alumno, docente, administrador.

- Interfaz de administración para gestionar usuarios y documentos del sistema.

- Evaluación automática de la calidad de las respuestas del sistema RAG.

- Migración del frontend a Next.js para mejorar el rendimiento con SSR.

12. Glosario

API REST — Interfaz de programación de aplicaciones que usa el protocolo HTTP para la comunicación entre sistemas mediante los métodos GET, POST, PUT y DELETE.

BCrypt — Algoritmo de cifrado utilizado para almacenar contraseñas de forma segura. Genera un hash irreversible que se compara en cada inicio de sesión.

ChromaDB — Base de datos vectorial utilizada para almacenar y consultar embeddings de documentos de forma eficiente.

Embedding — Representación numérica de un texto en forma de vector de alta dimensión que captura su significado semántico.

FastAPI — Framework de Python para crear APIs REST de alto rendimiento con documentación automática.

JWT (JSON Web Token) — Estándar para la transmisión segura de información entre sistemas mediante tokens firmados digitalmente.

LangChain — Librería de Python que facilita la construcción de aplicaciones basadas en modelos de lenguaje, incluyendo sistemas RAG.

LLM (Large Language Model) — Modelo de lenguaje de gran tamaño entrenado con enormes cantidades de texto. En este proyecto se usa GPT-3.5-turbo de OpenAI.

ORM (Object-Relational Mapping) — Técnica que permite trabajar con bases de datos relacionales usando objetos del lenguaje de programación. En este proyecto se usa Hibernate.

RAG (Retrieval-Augmented Generation) — Técnica de IA que combina la recuperación de información relevante de documentos con la generación de respuestas por un modelo de lenguaje.

Spring Boot — Framework de Java que simplifica el desarrollo de aplicaciones web y microservicios con configuración automática.

Token — Cadena de texto cifrada que identifica a un usuario autenticado y se incluye en cada petición HTTP para verificar su identidad.

Vector Store — Base de datos especializada en almacenar y buscar vectores de embeddings mediante similitud semántica.

13. Bibliografía

A continuación se listan las principales fuentes consultadas durante el desarrollo del proyecto:

[Spring Boot Documentation](#)

[Spring Security Reference](#)

[FastAPI Documentation](#)

[LangChain Python Documentation](#)

[OpenAI API Reference](#)

[React Documentation](#)

[PostgreSQL 14 Documentation](#)

[JWT Introduction](#)

[FAISS — Efficient Similarity Search \(Meta AI Research\)](#)

[Baeldung — Spring Boot Tutorials](#)

[Axios Documentation](#)